UNIVERSIDAD TECNOLÓGICA DE SAN JUAN DEL RÍO

ACTIVIDAD

INVESTIGACION

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL / DESARROLLO DE SOFTWARE MULTIPLATAFORMA / JALPAN

PRESENTA:

ESPINOZA HERNANDEZ CELESTE

PROFESOR DE ASIGNATURA

MARÍA DE MONSERRATH GUERRERO CHAVERO

JALPAN DE SERRA, QRO.

2 DE OCTUBRE DE 2026

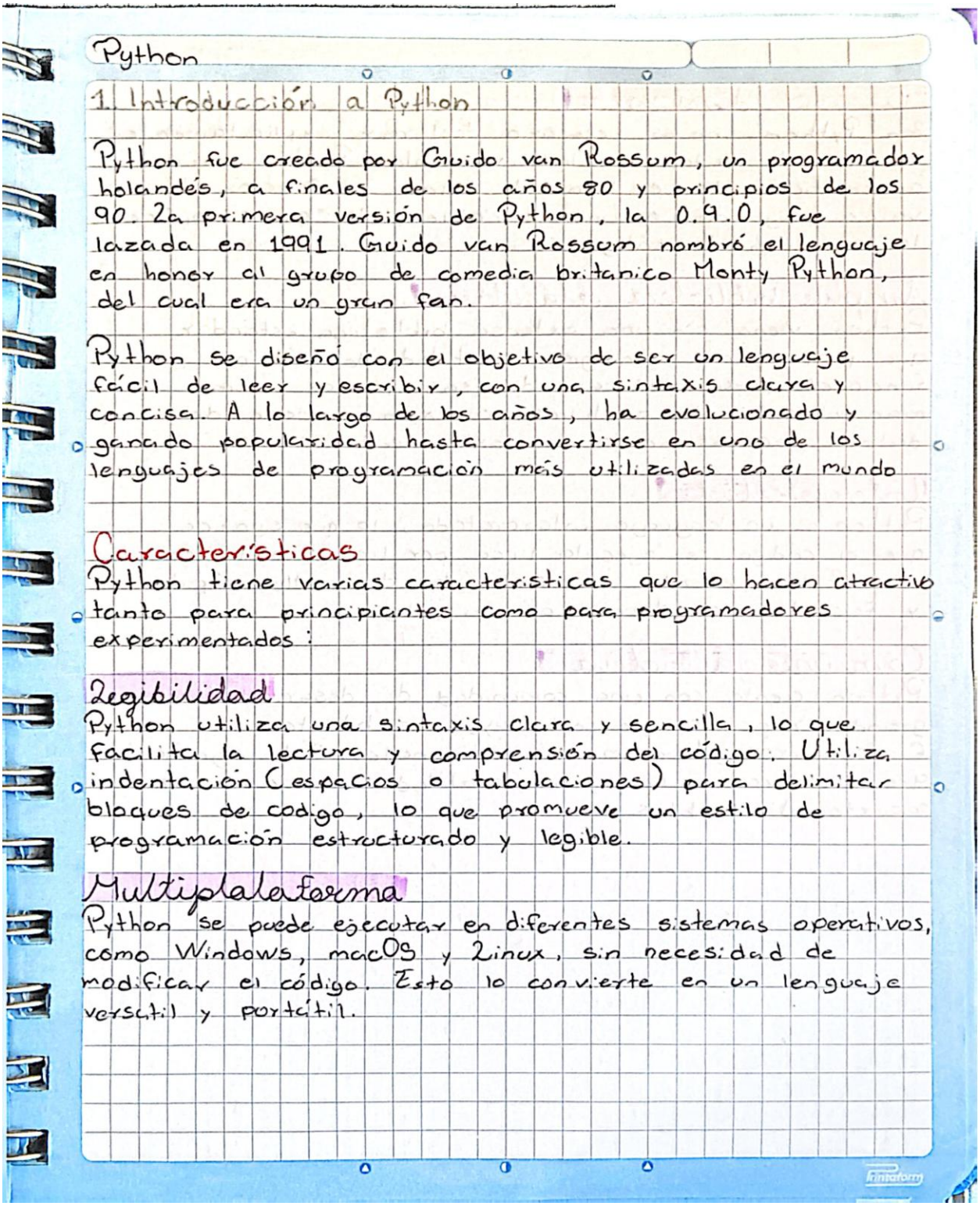
INDICE

INTRODUCCIÓN 3
APUNTES 4
CONCLUSIÓN 18

INTRODUCCIÓN

Python es un lenguaje de programación de alto nivel, reconocido por su sintaxis sencilla, clara y fácil de comprender, lo que lo convierte en una herramienta adecuada tanto para personas que comienzan a programar como para desarrolladores con experiencia. En el curso de Python de Santander Open Academy se presentan los conceptos fundamentales necesarios para comenzar a desarrollar programas y comprender la lógica de programación. Python puede utilizarse en diferentes áreas, como el desarrollo de aplicaciones y páginas web, el análisis de datos, la automatización de tareas, la inteligencia artificial y el desarrollo de software. A través del aprendizaje de sus elementos básicos, como variables, tipos de datos, operadores, estructuras de control, funciones y ciclos, es posible desarrollar programas de manera estructurada y resolver diferentes tipos de problemas mediante código.

APUNTES



Python

1. Introducción a Python

Python fue creado por Guido van Rossum, un programador holandés, a finales de los años 80 y principios de los 90. La primera versión de Python, la 0.9.0, fue lazada en 1991. Guido van Rossum nombró el lenguaje en honor al grupo de comedia británico Monty Python, del cual era un gran fan.

Python se diseñó con el objetivo de ser un lenguaje fácil de leer y escribir, con una sintaxis clara y concisa. A lo largo de los años, ha evolucionado y ganado popularidad hasta convertirse en uno de los lenguajes de programación más utilizadas en el mundo

Características

Python tiene varias características que lo hacen atractivo tanto para principiantes como para programadores experimentados:

Legibilidad

Python utiliza una sintaxis clara y sencilla, lo que facilita la lectura y comprensión del código. Utiliza indentación (espacios o tabulaciones) para delimitar bloques de codigo, lo que promueve un estilo de programación estructurado y legible.

Multiplataforma

Python se puede ejecutar en diferentes sistemas operativos, como Windows, macOS y Linux, sin necesidad de modificar el código. Esto lo convierte en un lenguaje versatil y portátil.

Tipado dinámico

En Python, no es necesario declarar explícitamente el tipo de datos de las variables. Python infiere automaticamente el tipo de datos en función del valor asignado a una variable, lo que simplifica la escritura de código.

Amplia biblioteca estándar

Python viene con una extensa biblioteca estándar que proporciona una gran cantidad de módulos y funciones para realizar diversas tareas, como manipulación de archivos, conexión a base de datos, procesamiento de texto, entre otros.

Interpretado

Python es un lenguaje interpretado, lo que significa que el código se ejecuta linea por linea en tiempo real. Esto permite un ciclo de desarrollo rápido y facilita la depuración del código

Comunidad activa

Python cuenta con una comunidad de desarrolladores grande y activa que contribuye con bibiotecas, frameworks y herramientas adicionales. Esto significa que encontrarás una gran cantidad de recursos y soporte disponibles.

Aplicaciones

Python se utiliza en una amplia gama de aplicaciones y dominios, algunos ejemplos son:

Desarrollo web

Python es ampliamente utilizado en el desarrollo web backend, con frameworks populares como Django y Flask.

Ciencia de datos

Python es el lenguaje preferido para el analisis de datos y la ciencia de datos debido a bibliotecas como NumPy, Pandas, Matplotlib.

Inteligencia artificial y machine learning

Python es la eleccion principal para proyectos de IA y machine learning gracias a bibliotecas como Tensorflow y Scikit-learn

Automatización de tareas

Python se puede utilizar para automatizar tareas repetitivas, como procesamiento de archivos, web scraping y pruebas de software.

Desarrollo de juego

Python se utiliza en el desarrollo de juegos simples, especialmente con bibliotecas como Pygame.

Conceptos básicos de la sintaxis en Python

Identación

En Python, la identación (espacios o tabulaciones al inicio de una línea) se utiliza para delimitar bloques de código. A diferencia de otros lenguajes que utilizan llaves o palabras clave, Python utiliza la identación para determinar el alcance de las declaraciones. Por ejemplo:

```
if condition:
    # Bloque de código si la condición es verdadera
    instrucción 1
    instrucción 2
else:
    # Bloque de código si la condicion es falsa
    instrucción 3
    instrucción 4
```

Comentarios

Los comentarios son líneas de texto en el código que son ignoradas por el interprete de Python. Se utilizan para explicar o documentar el codigo. En Python, los comentarios de una sola línea comienzan con el simbolo #, mientras que los comentarios de varias lineas se encierran entre triples comillas """. Por Ejemplo:

```
# Este es un comentario de una sola linea
"""
Este es un comentario
de varias líneas
"""
```

Sensibilidad a mayusculas y minusculas

Python distingue entre mayúsculas y minúsculas. Por lo tanto, variable, Variable y VARIABLE se consideran variables diferentes

Punto y coma

A diferencia de otros lenguajes, Python no requiere el uso de punto y coma (;) al final de cada instrucción. Sin embargo si deseas escribir varias instrucciones en una sola linea, pueden separarlas con un punto y coma. Por ejemplo:

```
instruccion1; instruccion2; instruccion3
```

Uso de paréntesis

Los paréntesis se utilizan para agrupar extensiones, definir funciones y realizar llamadas a funciones. Por ejemplo:

```
resultado = (a+b) * c
```

2 Fundamentos de python

Tipos de datos básicos

En Python, los tipos de datos básicos son las categorías en la que se pueden clasificar los valores que utilizamos en nuestros programas. Comprender los diferentes tipos de datos es fundamental para trabajar con variables y realizar operaciones en Python los tipos de datos básicos incluyen:

Enteros (int)

Los numeros enteros son aquellos que no tienen una parte decimal. Se representan simplemente escribiendo el numero sin comillas ni puntos decimales. Por ej:

```
edad = 25
cantidad = 100
```

Flotantes (float)

Los numeros flotantes, tambien conocidos como números de punto flotante, son aquellos que tienen una parte decimal. En Python se representan utilizando un punto para separar la parte entera de la parte decimal Por ejemplo:

```
precio = 9.99
altura = 1.75
```

Cadenas de texto (string)

Las cadenas de texto, o simplemente cadenas, son secuencias de caracteres encerradas entre comillas simples ('...') o dobles ("...") Se utilizan para representar texto en Python. Por ejemplo:

```
nombre = "María"
mensaje = '¡Hola, mundo!'
```

Puede incluir caracteres especiales en las cadenas utilizando el carácter de escape \. Por ejemplo, para incluir comillas dentro de una cadena, puedes usar \' o \". También puedes utilizar la notación de triple comilla (""..." o '''...''') para crear cadenas de varias líneas.

Booleanos

Los valores booleanos representan los valores de verdad: True (verdadero) y False (falso).
Se utilizan comúnmente en expresiones condicionales y operaciones lógicas. Por ejemplo

```
es_mayor_de_edad = True
tiene_descuento = False
```

2.1. Variables

Las variables son contenedores que nos permiten almacenar y manipular datos en nuestros programas. Pueden pensar en una variable como una etiqueta a la que asignas a un valor específico. En Python, no es necesario declarar el tipo de datos de una variable de antemano, ya que Python infiere el tipo de datos automáticamente en función del valor asignado.

Declaración y asignación de variables

Las variables son contenedores que nos permiten almacenar y manipular datos en nuestros programas. Para declarar y asignar un valor a una variable en Python, utilizamos el operador de asignación =. El nombre de la variable va a la izquierda del operador, y el valor que deseas asignar va a la derecha. Por ejemplo:

```
nombre = "Maria"
edad = 25
altura = 1.75
es_estudiante = True
```

En el ejemplo, hemos declarado y asignado valores a las variables nombre, edad, altura y es_estudiante. Python infiere automaticamente el tipo de datos de cada variable en función del valor asignado.

Tambien puedes asignar el mismo valor a múltiples variables en una sola linea utilizando el operador de asignación múltiple:

```
a = b = c = 10
```

En este caso, las variables a, b tendran el valor 10

Normas para nombrar variables

Al nombrar variables en Python, es importante seguir algunas normas para mantener un código legible y evitar errores:

Los nombres de las variables solo pueden contener letras (a-z, A-Z) números (0-9) y guiones bajos (_). No pueden comenzar con un número.

Python distingue entre mayúsculas y minúsculas, por lo que nombre y Nombre son variables diferentes.

No se pueden utilizar palabras clave reservadas de Python como nombres de variables (por ejemplo, if, else, for, while, etc)

Se recomienda utilizar nombres descriptivos para las variables, que indiquen claramente su propósito: edad, total_ventas, etc.

nombre_completo = curso + 01

2.2 Operadores

Los operadores son simbolos que permiten realizar operaciones con variables y valores. En Python se utilizan principalmente para hacer cálculos, comparar valores y combinar condiciones

Operadores aritmeticos

Sirven para realizar operaciones matematicas

Operador	Nombre	Ejemplo	Resultado
+	Suma	10 + 3	13
-	Resta	10 - 3	7
*	Multiplicacion	10 * 3	30
/	División	10 / 3	3.333...
//	División entera	10 // 3	3
%	Módulo (resto)	10 % 3	1
**	Exponenciación	10 ** 3	1000

Ejemplo

```
a = 10
b = 3

Suma = a + b
resta = a - b
multiplicacion = a * b
division = a / b
division_entera = a // b
modulo = a % b
potencia = a ** b
```

Operadores de comparación

Se utilizan para comparar dos valores y siempre devuelven un resultado booleano: True o False.

Operador	Significado	Ejemplo	Resultado
==	igual a	10 == 3	False
!=	Diferente de	10 != 3	True
>	Mayor que	10 > 3	True
<	Menor que	10 < 3	False
>=	Mayor o igual	10 >= 3	True
<=	Menor o igual	10 <= 3	False

Ejemplo

```
a = 10
b = 3

print (a==)    # False
print (a!=)    # True
print (a> )    # True
```

Operadores lógicos

Permiten combinar o negar condiciones

Operador	Función
and	True si ambas condiciones son verdaderas
or	True si al menos una condición es verdadera
not	Invierte el resultado de una condición

Ejemplo

```
a = 10
b = 3

resultado_and = (a > 5) and (b < 5)    # True
resultado_or = (a > 15) or (b < 5)     # True
resultado_not = not (a > 5)            # False
```

3. Estructuras de control en Python

Las estructuras de control permiten controlar el flujo de ejecución de un programa. Sirven principalmente para tomar decisiones y ejecutar diferentes instrucciones dependiendo de determinadas condiciones

Estructuras condicionales

Permiten ejecutar un bloque de código dependiendo de si una condición se cumple o no

Las principales son:

- if

Ejecuta un bloque de código si la condición es verdadera

Ejemplo

```
edad = 18

if edad >= 18
    print ("Eres mayor de edad.")
```

- if - else

permite elegir entre dos opciones

- if -> Cuando la condicion es verdadera
- else -> Cuando la condicion es falsa

Ejemplo:

```
edad = 15

if edad >= 18
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

Solo se ejecuta uno de los dos bloques

- if - elif - else

permite evaluar varias condiciones

- if -> primera condicion
- elif -> condiciones adicionales
- else -> se ejecuta si ninguna condicion se cumple

Ejemplo

```
calificación = 85

if calificacion >= 90:
    print("Exelente")
elif calificacion >= 80:
    print("Muy bueno")
elif calificacion
```

3.1 Bucles/loops

Los bucles permiten repetir un bloque de codigo varias veces. En python los principales son for y while

Bucle

Se utiliza para recorrer una secuencia o elemento iterable, como una lista, cadena, tupla o rango

Ejemplo

```
Frutas = ["manzana", "banana", "naranja"]

for fruta in frutas:
    print (fruta)
```

CONCLUSIÓN

En conclusión, Python es un lenguaje de programación versátil y accesible que permite desarrollar soluciones para una gran variedad de necesidades tecnológicas. El aprendizaje de sus conceptos fundamentales proporciona las bases necesarias para comprender cómo funciona un programa y desarrollar una adecuada lógica de programación. Además, su sintaxis sencilla facilita la escritura y comprensión del código, permitiendo que los conocimientos adquiridos puedan aplicarse posteriormente en proyectos más complejos. Por esta razón, conocer Python representa una herramienta importante para la formación de un desarrollador de software, ya que sus aplicaciones abarcan diferentes áreas de la tecnología y permiten continuar avanzando hacia temas como el desarrollo web, análisis de datos, automatización e inteligencia artificial.